Stein, saks, papir

Bygg et spill mot datamaskinen – steg for steg

Navn: _____ Klasse: _____

Slik bruker du heftet

I dette heftet lager du spillet stein, saks, papir. Du spiller mot datamaskinen, som vi kaller **CPU**. Du velger ved å trykke på en knapp, og CPU-en velger tilfeldig. Så finner programmet ut hvem som vant.

Du bruker nesten bare det du kan fra før: variabler, knapper, funksjoner, `textContent`, `${ }` og `if` / `else`. Spillet følger oppskriften **Hent – Regn – Vis** fra hefte 1: knappene og CPU-en gir oss valgene (hent), `if` finner vinneren (regn), og `textContent` viser resultatet (vis). Skriv av koden selv, lagre, og **test etter hvert steg**.

Lag filen `stein-saks-papir.html` i `JS`-mappen.

Slik skal spillet se ut

```
+--------------------------------------------------+
|                Stein, saks, papir                |
|            Velg ett av alternativene:            |
|           [Stein]    [Saks]    [Papir]           |
|                 Du valgte stein.                 |
|                 CPU valgte saks.                 |
|                     Du vant!                     |
+--------------------------------------------------+
```

Reglene

Du velger	Det slår	Fordi
Stein	Saks	Stein knuser saks.
Saks	Papir	Saks klipper papir.
Papir	Stein	Papir pakker inn stein.

Velger dere det samme, blir det uavgjort.

Nytt i dette heftet – bare fire små ting:

Kode	Hva den gjør
`spillRunde();`	En funksjon kan kalle en annen funksjon, akkurat som en knapp kan.
`Math.random()`	Gir et tilfeldig tall mellom 0 og 1, for eksempel `0.4718`.
`else if ( ... )`	«Ellers, hvis». Brukes når det er flere enn to muligheter.
`&&`	Betyr «og». Begge sjekkene må stemme.

Steg 1 – Skjelettet og CSS-en

Skriv `!` og trykk Tab for å få skjelettet. Endre `<title>`, og lag et `<style>`-element rett over `</head>`:

```
  <title>Stein, saks, papir</title>
  <style>
    /* CSS-en under skal stå her */
  </style>
</head>
```

Skriv denne CSS-en inni `<style>`. Rekkefølgen på reglene spiller ingen rolle. CSS-en gjør spillet penere, og du har lært alt sammen i CSS-heftene.

```
body {
  font-family: Arial, sans-serif;
  text-align: center;
  background-color: #f0f4f8;
}
.knapper {
  display: flex;
  justify-content: center;
  gap: 10px;
}
#txtResultat {
  font-size: 28px;
  font-weight: bold;
}
```

```
button {
  font-size: 20px;
  padding: 15px 25px;
  border: none;
  border-radius: 10px;
  background-color: #1f3a5c;
  color: white;
}
button:hover {
  background-color: #3d6a9e;
}
```

Steg 2 – HTML-en: knapper og tomme avsnitt

Skriv dette inni `<body>`:

```
  <h1>Stein, saks, papir</h1>
  <p>Velg ett av alternativene:</p>

  <div class="knapper">
    <button onclick="velgStein()">Stein</button>
    <button onclick="velgSaks()">Saks</button>
    <button onclick="velgPapir()">Papir</button>
  </div>

  <p id="txtSpiller"></p>
  <p id="txtCpu"></p>
  <p id="txtResultat"></p>
```

Lagre og se på siden. Knappene står på rad og blir lysere når du holder musen over dem. Hvilken CSS-regel gjør at de står på rad?

Svar: _____

De tre avsnittene er tomme. Der skal vi skrive hva du valgte, hva CPU-en valgte og hvem som vant.

Steg 3 – Få én knapp til å virke

Start alltid i det små. Skriv en `<script>`-blokk **nederst** i `<body>`, rett over `</body>`:

```
<script>
  function velgStein() {
    txtSpiller.textContent = "Du valgte stein.";
  }
</script>
```

Lagre og trykk på Stein. Står det «Du valgte stein.»? Da henger knappen, funksjonen og avsnittet sammen.

Steg 4 – Tre knapper og én felles funksjon

Vi kunne skrevet hele spillet tre ganger, én gang i hver knappefunksjon. Men da må vi rette alle feil tre ganger! I stedet gjør hver knapp bare to ting: den **husker** valget ditt i variabelen `spiller`, og den **kaller** funksjonen `spillRunde()`, som gjør resten.

Bytt ut alt inni `<script>` med dette:

```
  let spiller = "";

  function velgStein() {
    spiller = "stein";
    spillRunde();
  }

  function velgSaks() {
    spiller = "saks";
    spillRunde();
  }

  function velgPapir() {
    spiller = "papir";
    spillRunde();
  }

  function spillRunde() {
    txtSpiller.textContent = `Du valgte ${spiller}.`;
  }
```

- `let spiller = "";` står **utenfor** funksjonene, slik `saldo` gjorde i sparegrisen. Da kan alle funksjonene bruke den. `""` betyr tom tekst.
- `spillRunde();` kaller funksjonen. Det er det samme som `onclick="spillRunde()"` gjør – bare fra JavaScript.

Test alle tre knappene.

Undersøk: Slett linjen `spillRunde();` i `velgSaks`. Trykk på Saks. Hva skjer – og hvorfor? Sett linjen tilbake etterpå.

Svar: _____

Steg 5 – CPU-en velger tilfeldig

5a – Se på et tilfeldig tall. Legg til to linjer nederst i `spillRunde`:

```
    let tall = Math.random();
    txtCpu.textContent = `Det tilfeldige tallet er ${tall}`;
```

`Math.random()` gir et nytt tilfeldig tall mellom 0 og 1 hver gang (det kan bli 0, men aldri helt 1). Trykk på en knapp fem ganger, og skriv ned de første fire sifrene i tallet hver gang.

Trykk	1	2	3	4	5
Tallet					

5b – Gjør tallet om til stein, saks eller papir. Vi deler tallene fra 0 til 1 i tre omtrent like store biter:

```
0             0.33            0.66              1
|---------------|---------------|-----------------|
      stein            saks             papir
```

Bytt ut de to linjene fra 5a med dette:

```
    let tall = Math.random();
    let cpu = "";
    if (tall < 0.33) {
      cpu = "stein";
    } else if (tall < 0.66) {
      cpu = "saks";
    } else {
      cpu = "papir";
    }
    txtCpu.textContent = `CPU valgte ${cpu}.`;
```

- Datamaskinen sjekker ovenfra og ned, og stopper ved den **første** sjekken som stemmer.
- `else if` betyr «ellers, hvis». Den blir bare sjekket hvis sjekken over **ikke** stemte.
- `let cpu = "";` lager variabelen **før** `if`. Inni `if` gir vi den en ny verdi.

Forutsi: Hva velger CPU-en hvis tallet blir slik? Tenk ovenfra og ned! Test så spillet mange ganger.

Tallet	`0.12`	`0.50`	`0.97`	`0.33`	`0.6599`
CPU velger					

Steg 6 – Hvem vant?

Det er tre måter du kan **vinne** på (se reglene på side 1). Valgte dere likt, er det uavgjort. Ellers har CPU-en vunnet.

Forutsi: Fyll ut tabellen med **V** (du vant), **T** (du tapte) eller **U** (uavgjort) før du skriver koden.

Du ↓ CPU →	stein	saks	papir
stein			
saks			
papir			

Legg til den nye delen nederst i `spillRunde`. Slik ser hele funksjonen ut etterpå:

```
function spillRunde() {
  txtSpiller.textContent = `Du valgte ${spiller}.`;

  // CPU-en velger (fra steg 5)
  let tall = Math.random();
  let cpu = "";
  if (tall < 0.33) {
    cpu = "stein";
  } else if (tall < 0.66) {
    cpu = "saks";
  } else {
    cpu = "papir";
  }
  txtCpu.textContent = `CPU valgte ${cpu}.`;

  // Hvem vant? (NYTT – skriv inn alt herfra og ned)
  let resultat = "";
  if (spiller == cpu) {
    resultat = "Uavgjort!";
  } else if (spiller == "stein" && cpu == "saks") {
    resultat = "Du vant!";
  } else if (spiller == "saks" && cpu == "papir") {
    resultat = "Du vant!";
  } else if (spiller == "papir" && cpu == "stein") {
    resultat = "Du vant!";
  } else {
    resultat = "CPU vant!";
  }
  txtResultat.textContent = resultat;
}
```

- `&&` betyr «og». `spiller == "stein" && cpu == "saks"` stemmer bare hvis du valgte stein **og** CPU-en valgte saks.
- Vi trenger ikke skrive alle måtene CPU-en kan vinne på. Er det ikke uavgjort, og du ikke har vunnet, må CPU-en ha vunnet. Det tar `else` seg av.

Test smart. Det er vanskelig å teste alle ni kombinasjonene når CPU-en velger tilfeldig. Jukse-tips: Legg til linjen `cpu = "saks";` rett **under** hele `if`-blokken fra steg 5 (over linjen med `txtCpu`). Nå velger CPU-en alltid saks. Test de tre knappene, bytt til `"stein"` og `"papir"`, og sett en hake i tabellen på forrige side for hver kombinasjon som stemmer. **Slett juksekoden når du er ferdig!**

Undersøk: Skriv `"Stein"` med stor S i `velgStein`. Hva skjer når du spiller stein? Hvorfor?

Svar: _____

Utvidelser

Spillet virker! Nå kan du gjøre det bedre. Velg oppgaver selv. Én stjerne er lettest. Lagre en kopi av filen før du begynner, så har du noe som virker hvis du roter det til.

★ **1. Poengtavle.** Lag to variabler **utenfor** funksjonene: `let spillerPoeng = 0;` og `let cpuPoeng = 0;`. Øk riktig variabel med `++` i `if`-ene der noen vinner. Vis stillingen i et nytt avsnitt: «Du 3 – 2 CPU».

- [ ] Poengene øker riktig.
- [ ] Stillingen vises etter hver runde.

★ **2. Farge på resultatet.** Grønn tekst når du vinner, rød når du taper og grå ved uavgjort. Hint: Husker du `document.body.style = "background-color:blue;"`? Prøv `txtResultat.style = "color: green;"` inni `if`-ene.

★ **3. Nullstill.** Lag en knapp «Ny kamp» som setter begge poengene til 0 og tømmer alle avsnittene.

★★ **4. Antall runder.** Tell hvor mange runder som er spilt, og vis det: «Runde 7».

★★ **5. Først til 5.** Når en av dere får 5 poeng, skal det stå «Du vant kampen!» eller «CPU vant kampen!». Hint: `if (spillerPoeng == 5) { ... }`

★★ **6. Symboler.** Bytt ut ordene med emojier: ✊ ✌ ✋. På Windows åpner du emoji-panelet med **Windows-tasten + punktum**. Bruk dem både på knappene og når du viser valgene. Gjør symbolene store med CSS.

★★ **7. Er CPU-en rettferdig?** Tell hvor mange ganger CPU-en velger stein, saks og papir. Spill 30 runder og fyll ut:

Stein	Saks	Papir

Er det omtrent like mange av hver? Hvorfor ikke helt likt?

Svar: _____

★★★ **8. Stein, saks, papir, øgle, Spock.** Utvid spillet til fem valg. Nå må du dele tallet fra `Math.random()` i fem biter, og det finnes ti måter å vinne på:

- Saks klipper papir, papir pakker inn stein, stein knuser øgle, øgle forgifter Spock, Spock knuser saks
- Saks halshugger øgle, øgle spiser papir, papir motbeviser Spock, Spock fordamper stein, stein knuser saks

Husk: Test etter hvert steg – og bruk juksetipset fra steg 6 når du vil teste en bestemt kombinasjon.